

수치 데이터 처리

Jinu Gong

강의 안내

* 주차 별 수업계획은 진행에 따라 변경될 수 있음

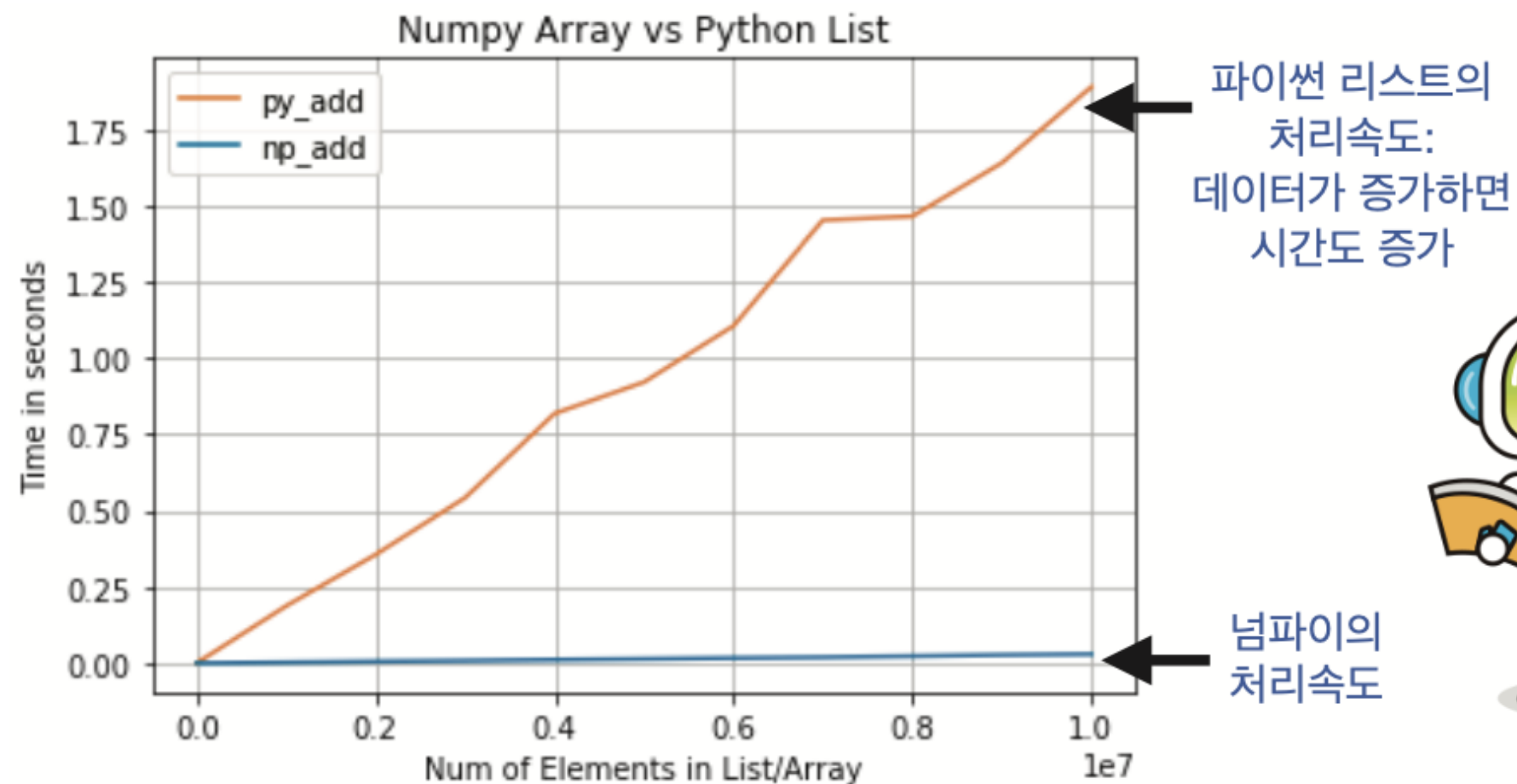
주차	날짜	온라인 강의내용	오프라인 강의내용
9	5/4	Numpy	Numpy와 파이썬 복습, 조편성
10	5/11	생성형 AI 기초 - 생성 모델 기초 - 대형 언어모델	Hugging face를 통한 공개 모델 활용
11	5/18	LLM과 토큰나이저 개념 - 대형 언어모델의 등장과 발전 - 대형 언어모델의 동작 기초	간단한 챗봇 만들기
12	5/25	인공지능 윤리 2 - 생성형 AI의 위험한 사용 사례 - AI가 틀렸을 때의 위험 - RAG와 그 필요성	휴강
13	6/1	사전학습과 미세조정 RAG	실습으로 알아보는 RAG
14	6/8	Agent AI와 바이트코딩	9~13주차 수업내용 퀴즈 Tool을 통한 간단한 Agent 직접 만들기
15	6/15	기말 프로젝트 발표	기말프로젝트 조별 활동
16	6/22	기말 프로젝트 발표	

학습 목표

- 넘파이를 사용하는 이유를 알아보자
- 넘파이가 제공하는 다차원 배열의 속성에 대하여 알아보자
- 넘파이의 강력한 기능을 직접 사용해 보며 익혀보자
- 넘파이로 각종 확률 분포에서 난수를 생성해 데이터를 생성해보자
- 고차원 배열의 인덱싱 기법에 대해서도 알아보자
- 넘파이가 제공하는 데이터 분석 함수를 알아보자
- 다수 변수 간의 상관관계를 계산해보자

넘파이를 사용하는 이유

- 넘파이는 대용량의 배열과 행렬 연산을 빠르게 수행하며, 고차원적인 수학 연산자와 함수를 포함하고 있는 파이썬 라이브러리이다.
- 표에서 알 수 있듯이 넘파이의 배열은 주황색으로 표시된 파이썬의 리스트에 비하여 **처리속도가 매우 빠름**을 알 수 있다.



파이썬 리스트의
처리속도:
데이터가 증가하면
시간도 증가

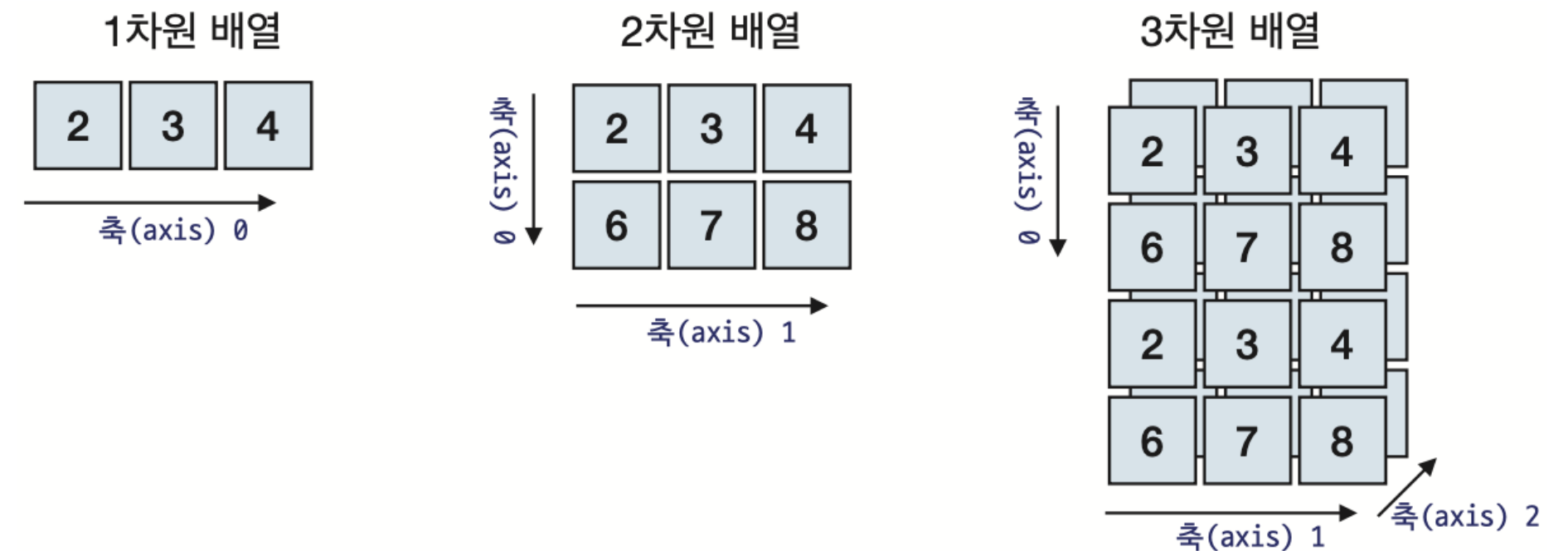
넘파이의
처리속도



넘파이 수치 데이터의 연산
은 리스트 수치 데이터의 연
산보다 처리 속도가 훨씬 빠
릅니다.

넘파이를 사용하는 이유

- 데이터 분석이나 기계 학습 프로젝트를 수행한다면 넘파이에 대한 확실한 이해가 필수적이다.
- 데이터 분석을 위한 패키지인 **판다스**Pandas나 기계학습을 위한 Scikit-learn, Tensorflow 등이 넘파이 위에서 작동한다.
- 넘파이의 핵심적인 객체는 다차원 배열이다. 예를 들어서 정수들의 2차원 배열(테이블)을 넘파이를 이용해서 생성할 수 있다. 배열의 각 요소는 **인덱스**index라고 불리는 정수들로 참조된다.
- 넘파이에서 차원은 **축**axis이라고도 한다.



리스트와 넘파이 배열의 차이

- 데이터 과학자들은 왜 넘파이를 많이 사용할까?
- 넘파이는 성능이 우수한 ndarray 객체를 제공한다.
- 전통적으로 **배열**array은 동일한 자료형을 가진 데이터를 연속적으로 저장한다.
- 파이썬의 리스트는 동일하지 않은 자료형을 가진 항목들을 담을 수 있다.
- ndarray의 장점을 정리하면 아래와 같다.

- ndarray는 C 언어에 기반한 배열 구조이므로 메모리를 적게 차지하고 속도가 빠르다.
- ndarray를 사용하면 배열과 배열 간에 수학적인 벡터 연산을 이용할 수 있다.
- ndarray는 고급 연산자와 풍부한 함수들을 제공한다.

리스트와 넘파이 배열의 차이

- 넘파이 배열의 장점을 이해하기 위한 간단한 예제를 살펴보자. 다음과 같이 학생들의 중간고사와 기말고사 성적을 저장하고 있는 리스트가 있다고 하자.

```
mid_scores = [10, 20, 30]    # 파이썬 리스트 mid_scores
final_scores = [70, 80, 90]  # 파이썬 리스트 final_scores
```

- Mid scores는 학생 3명의 중간고사 성적을 저장하고 final scores은 기말고사 성적을 저장하고 있는데, 다음 표와 같이 각 학생들의 중간고사 성적과 기말고사 성적을 합하여 오른쪽의 총점(total)이라는 리스트를 만들고 싶다.

	중간고사 성적	기말고사 성적	총점
학생 #1	10	70	80
학생 #2	20	80	100
학생 #3	30	90	120

리스트와 넘파이 배열의 차이

- 그런데 파이썬 리스트 더하기 연산자는 두 리스트를 연결하므로, `mid_scores + final_scores`을 적용하면 다음과 같이 2개의 리스트를 연결한 리스트가 만들어진다.

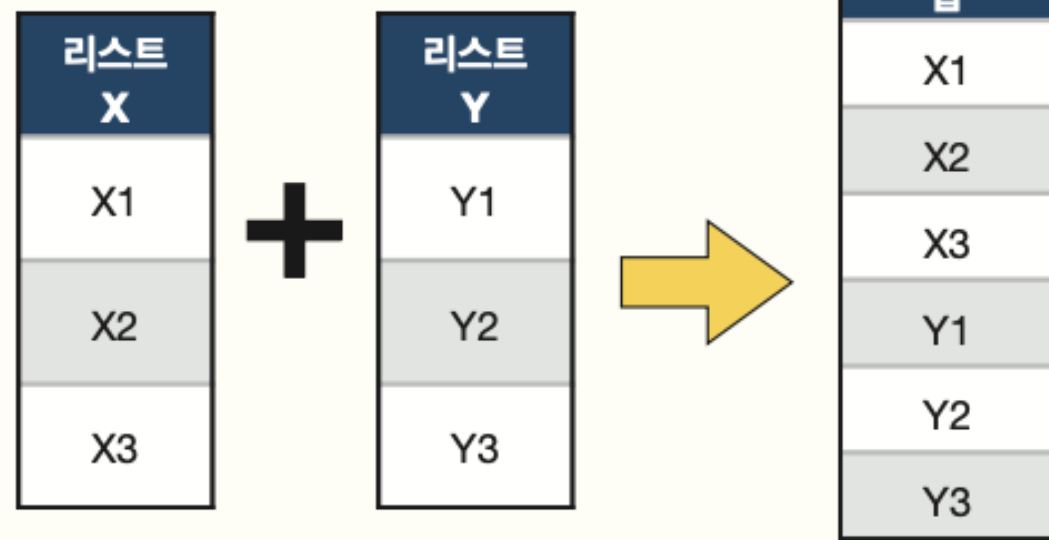
```
>>> total = mid_scores + final_scores    # 원소간의 합이 아닌 리스트를 연결함
>>> total
[10, 20, 30, 70, 80, 90]
```

- 이것은 분명히 우리가 원하는 연산이 아니다.
- 하지만 넘파이 배열에 +연산을 하면, 대응되는 값끼리 합쳐진 결과를 얻을 수 있다.

리스트와 넘파이 배열의 차이

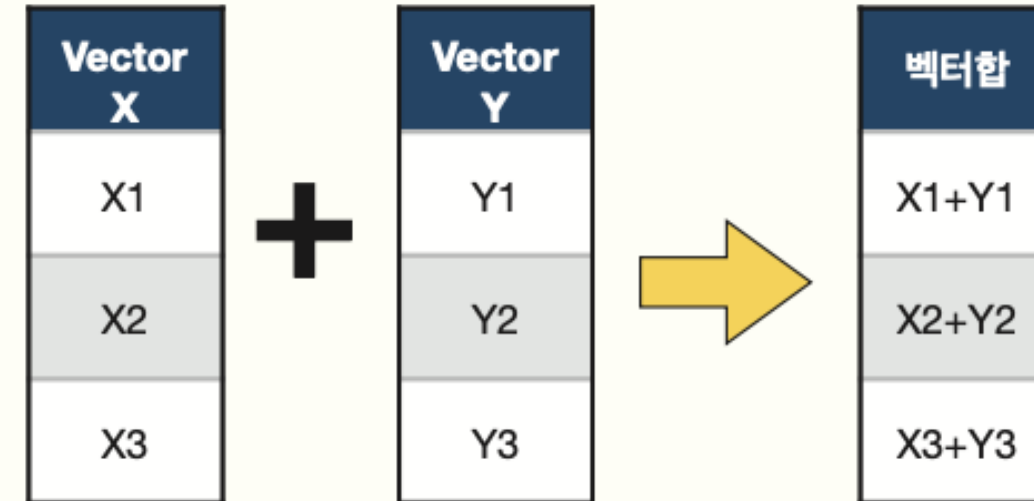
- 이와 같이 넘파이의 덧셈 연산은 차원이 같은 두 벡터의 원소들끼리 덧셈을 한다. 이를 **벡터합 연산**이라고 한다.

파이썬 기본 리스트 연산



파이썬 리스트의 덧셈 연산 X+Y와 그 결과

넘파이의 다차원 배열 연산



넘파이 다차원 배열의 덧셈 연산 X+Y와 그 결과

넘파이의 다차원 배열끼리는 덧셈을 어떻게 할까요? 이 경우 차원이 같은 두 벡터의 원소들끼리 덧셈을 합니다. 과학자들은 이와 같은 벡터합 연산을 훨씬 더 많이 사용합니다.

넘파이의 별칭과 간단한 배열 연산

- 파이썬에서 넘파이를 사용하려면, 다음과 같이 넘파이 패키지를 불러와야 한다.
- `import ~ as`에서 `as` 뒤에 나타나는 이름은 `as` 앞의 이름을 대체하는 별칭이다. 보통 numpy의 **별칭**^{alias}으로 **np**를 사용한다.
- 앞으로 사용하게 될 넘파이 사용 프로그램에서 이 코드의 호출은 필수이다.

```
import numpy as np
```

- 넘파이 배열을 만들려면 넘파이가 제공하는 `array()` 함수를 이용한다. `array()` 함수에 파이썬 리스트를 전달하면 넘파이 배열이 생성된다.

```
mid_scores = np.array([10, 20, 30])
```

넘파이의 별칭과 간단한 배열 연산

```
mid_scores = np.array([10, 20, 30])    # 넘파이 ndarray 객체를 생성
final_scores = np.array([60, 70, 80])  # 넘파이 ndarray 객체를 생성
```

- 두 배열을 합하여 학생들의 총점을 계산해보자. 이번에는 ndarray의 덧셈 연산을 수행한다.

mid_scores	10	20	30
	+		
final_scores	60	70	80
	=		
total	70	90	110



넘파이의 + 연산자는
2개의 넘파이 배열에서 대응
되는 원소들끼리 합을 구해
같은 크기의 배열에 담아요

```
total = mid_scores + final_scores
print('시험성적의 합계 :', total)    # 각 요소별 합계가 나타난다
print('시험성적의 평균 :', total/2)  # 평균을 얻기위해 모든 요소를 2로 나눈다.
```

```
시험성적의 합계 : [ 70  90 110]
시험성적의 평균 : [35. 45. 55.]
```

다차원 배열을 알아보자

- 넘파이의 핵심이 되는 **다차원 배열** `ndarray`은 다음과 같은 속성을 가지고 있다.
- 이러한 속성을 이용하여 프로그램의 오류를 찾거나 배열의 상세한 정보를 손쉽게 조회할 수 있다.

```
>>> a = np.array([1, 2, 3])          # 넘파이 ndarray 객체의 생성
>>> a.shape                          # a 객체의 형태(shape)
(3,)
>>> a.ndim                          # a 객체의 차원
1
>>> a.dtype                         # a 객체 내부 자료형
dtype('int32')
>>> a.itemsize                     # a 객체 내부 자료형이 차지하는 메모리 크기(byte)
4
>>> a.size                         # a 객체의 전체 크기(항목의 수)
3
```

다차원 배열을 알아보자

- 각각의 속성에 관련한 상세한 설명은 다음 표와 같다.

속성	설명
ndim	배열 축 혹은 차원의 개수
shape	배열의 차원으로 (m, n) 형식의 튜플 형이다. 이 때, m과 n은 각 차원의 원소의 크기를 알려주는 정수
size	배열 원소의 개수이다. 이 개수는 shape 내의 원소의 크기의 곱과 같다. 즉 (m, n) 형태 배열의 size는 $m \cdot n$ 이다.
dtype	배열 내의 원소의 형을 기술하는 객체이다. 넘파이는 파이썬 표준 자료형을 사용할 수 있으나 넘파이 자체의 자료형인 bool_, character, int_, int8, int16, int32, int64, float, float8, float16_, float32, float64, complex_, complex64, object 형을 사용할 수 있다.
itemsize	배열내의 원소의 크기를 바이트 단위로 기술한다. 예를 들어 int32 자료형의 크기는 $32/8 = 4$ 바이트가 된다.
data	배열의 실제 원소를 포함하고 있는 버퍼
stride	배열 각 차원별로 다음 요소로 점프하는 데에 필요한 거리를 바이트로 표시한 값을 모은 튜플

다차원 배열을 알아보자



잠깐 - 네이처에 소개된 넘파이

2020년 9월 저명한 과학 저널 **네이처Nature**에 넘파이에 대한 리뷰 논문이 게재되었다. 기초과학 분야 논문을 주로 게재하는 네이처에 소프트웨어 모듈이 소개되는 것은 매우 이례적인 일이다. 논문은 **텐서tensor**라 불리는 다차원 배열을 효율적으로 다루는 넘파이가 과학 분야에 파이썬 생태계를 제공했고, 과학 각 분야의 수치 데이터 처리 기술들이 상호운용성을 가질 수 있도록 하는 역할을 했다고 밝히고 있다. 블랙홀의 그림자를 최초로 찍어낸 **이벤트 호라이즌Event Horizon** 연구팀이 넘파이 배열을 이용해 연구의 각 단계별 데이터를 저장하고 다루었던 사례도 소개하였다.

영국 왕립 천문학회는 넘파이를 바탕으로 만들어진 **Astropy**에게 상을 수여하면서 "Astropy 프로젝트는 수백 명의 젊은 과학자들에게 버전 제어, 단위 검사, 코드 리뷰, 이슈 트래킹과 같은 전문가 수준의 소프트웨어 개발 경험을 제공했다. 현대의 연구자에게 이것들은 핵심적 기술들인데 물리학이나 천문학 분야의 정규 대학 교육에서는 종종 누락되고 있다."라고 밝혔다.

넘파이 배열의 연산

- 넘파이 배열에는 $+$ 연산자나 $*$ 연산자와 같은 수학적인 연산자를 얼마든지 적용할 수 있다. 예를 들어서 어떤 회사가 좋은 성과를 거두어 전 직원의 월급을 100만원씩 올려주기로 하였다. 어떻게 하면 될까? 현재 직원들의 월급이 [220, 250, 230]이라고 하자. 이것을 넘파이 배열에 저장한다.

```
import numpy as np  
  
salary = np.array([220, 250, 230])
```

- 넘파이 배열에 저장된 모든 값에 100을 더하려면 다음과 같이 배열에 스칼라 값 100을 더하면 된다.

```
salary = salary + 100  
print(salary)
```

```
[320, 350, 330]
```


넘파이 배열의 연산

- 위의 코드와 같이 넘파이 배열 salary에 100을 더하면 salary 배열의 모든 요소에 100이 더해진다. 만일 모든 직원들의 월급을 2배 올려주려면 어떻게 하면 될까? 다음과 같이 넘파이의 배열에 2를 곱하면 된다.

```
salary = np.array([220, 250, 230])  
salary = salary * 2  
print(salary)
```

```
[440, 500, 460]
```

- 물론 곱셈연산은 2.1과 같은 실수 값을 적용할 수도 있다.

```
salary = np.array([220, 250, 230])  
salary = salary * 2.1  
print(salary)
```

```
[462. 525. 483.]
```


넘파이 배열의 연산

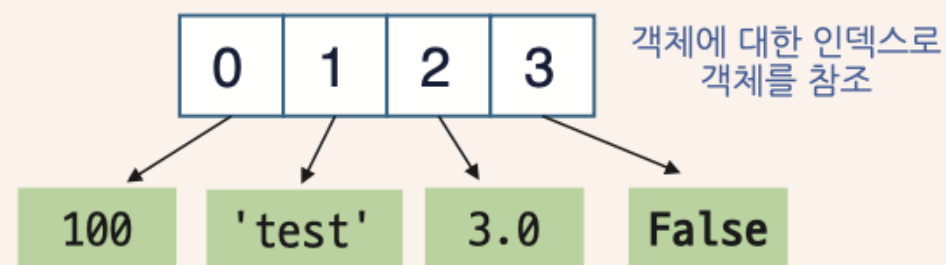
- 넘파이가 다차원 배열에 관한 계산을 빠르게 할 수 있는 이유를 알아보자.
- 파이썬의 리스트는 아래의 코드와 같이 다양한 자료형의 값을 가질 수 있다.
하지만 넘파이는 각 배열마다 타입을 하나만 가질 수 있다.
- 넘파이의 배열 안에는 동일한 타입의 데이터만 저장할 수 있다.

```
>>> lst = [ 100, 'test', 3.0, False] # 리스트는 다양한 자료형의 데이터를 가질 수 있다
>>> tangled = np.array([ 100, 'test', 3.0, False])
>>> print(tangled) # 넘파이는 다양한 자료형의 데이터를 하나의 자료형으로 만들어 저장한다
['100' 'test' '3.0' 'False']
```

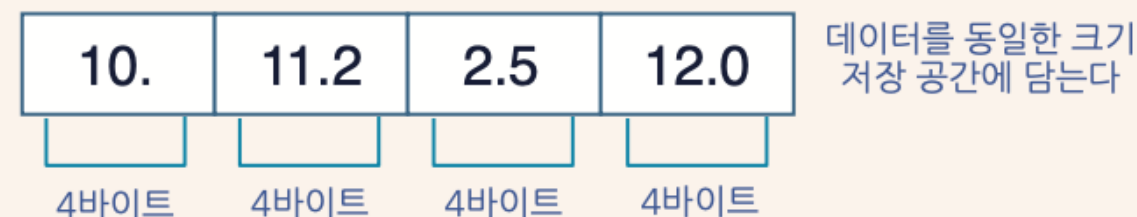
넘파이 배열의 연산

- 동일한 자료형으로만 데이터를 저장하면 각각의 데이터 항목에 필요한 저장 공간이 일정하다.
- 몇 번째 위치에 있는 항목이든 그 순서만 안다면 바로 접근할 수 있기 때문에 빠르게 데이터를 다룰 수 있는 것이다.
- 이렇게 원하는 위치에 바로 접근하여 데이터를 읽고 쓰는 일을 **임의 접근 random access**이라고 한다.

리스트가 데이터를 저장하는 방식



넘파이가 데이터를 저장하는 방식



벡터화 연산

- 넘파이의 장점인 벡터화 연산을 좀 더 깊이 살펴보자.
- 우선 코드의 수행시간을 측정하기 위한 time모듈에 대해 살펴보자. 파이썬의 time 모듈에 있는 time() 함수는 현재 컴퓨터의 시간을 밀리 초 단위로 알아낼 수 있다.
- 따라서 특정한 코드의 시작 전과 시작 후에 시간을 저장한 후 이 값을 빼는 방식으로 코드의 수행시간을 알아내는데 매우 유용하다.

```
import time

start = time.time() # 시작 시간을 저장
# 벡터화 연산 대신 반복문을 사용하자.
total = 0
for i in np.arange(100000):
    total = i + total
end = time.time() # 종료 시간을 저장

print('total =', total)
print("{:.5f} sec".format(end-start)) # 수행시간 출력
```

```
total = 4999950000
0.02376 sec
```

벡터화 연산

- 수행결과를 살펴보면 이 연산을 수행하는데 0.023초가 걸렸음을 알 수 있다. 이제는 넘파이의 벡터화 연산을 사용하는 `np.sum()`으로 동일한 결과를 만들어 보자.
- 벡터화 연산을 사용한 이 코드는 0.00065초 밖에 걸리지 않는다. 즉 이전의 코드에 비해서 수십 배 이상 더 빠르게 수행되는 것을 볼 수 있다. 따라서 넘파이의 성능을 최대한으로 끌어올리기 위해서는 가급적 `for` 루프를 사용하지 말고 **벡터화 연산 기능을 사용하는 것이 더 나은 방법**이다.

```
start = time.time() # 시작 시간을 저장
# 넘파이의 벡터화 연산을 사용하자
total = np.sum(np.arange(100000))
end = time.time() # 종료 시간을 저장
print('total =', total)
print("{:.5f} sec".format(end-start)) # 수행시간 출력
```

```
total = 4999950000
0.00065 sec
```

이전 코드에 비하여
매우 빠른 수행 속도

인덱싱과 슬라이싱

- 지금부터는 다음과 같은 성적이 저장된 1차원 배열에서 요소들을 꺼내는 방법을 살펴보자.

```
>>> scores = np.array([88, 72, 93, 94, 89, 78, 99])
```

- 넘파이 배열에서 특정한 요소를 추출하려면 인덱스를 사용한다.
- 파이썬 리스트와 마찬가지로 인덱스는 0부터 시작한다.
- 따라서 인덱스로 2를 지정하면 93이 출력된다.

```
>>> scores[2]  
93
```

- 마지막 요소에 접근하려면 리스트와 마찬가지로 인덱스로 -1을 주면 된다.

```
>>> scores[-1]  
99
```


인덱싱과 슬라이싱

인덱스	0	1	2	3	4	5	6
scores	88	72	93	94	89	78	99
음수 인덱스	-7	-6	-5	-4	-3	-2	-1

```
>>> scores[2]  
93
```

인덱싱은 특정한 요소를 얻는 방법이다

인덱스	0	1	2	3	4	5	6
scores	88	72	93	94	89	78	99
음수 인덱스	-7	-6	-5	-4	-3	-2	-1

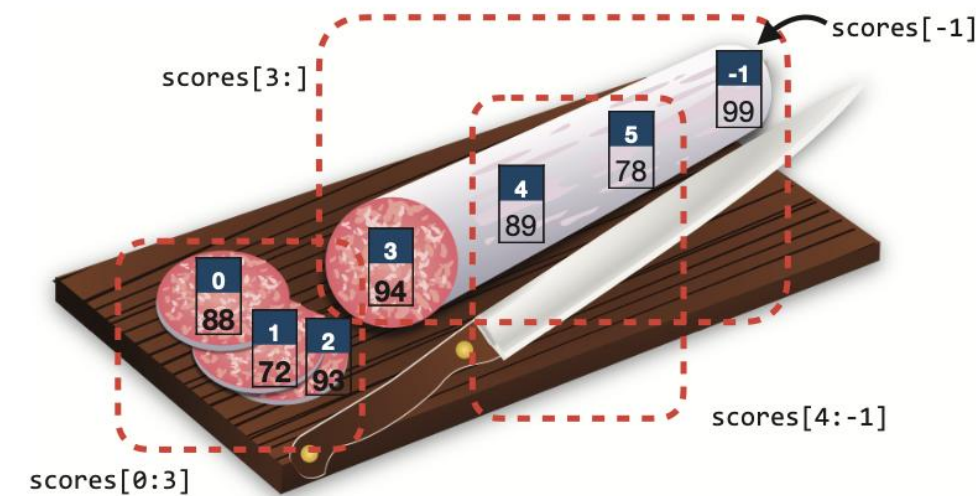
```
>>> scores[0:3]  
[88, 72, 93]
```

슬라이싱은 요소 집합을 얻는 방법이다

인덱싱과 슬라이싱

- 넘파이 배열에서는 다음과 같이 슬라이싱도 가능하다. 아래 코드와 같이 마지막 항목 인덱스가 3 일 때는 3-1인 `scores[2]` 항목까지 슬라이싱 된다는 것에 주의하도록 하자.

```
>>> # 첫 번째, ... 세 번째 항목 슬라이싱
>>> scores[0:3]
array([88, 72, 93])
>>> # 두 번째, ... 네 번째 항목 슬라이싱
>>> scores[1:4]
array([72, 93, 94])
```



- 또한 다음과 같이 시작 인덱스나 종료 인덱스는 생략이 가능하다. 이것은 파이썬의 리스트와 동일하다. 또한 `scores[4:-1]`과 같은 음수 인덱싱도 가능하다.

```
>>> scores[:3]      # 첫 인덱스를 생략하면 디폴트 값은 0임
array([88, 72, 93])
>>> scores[3:]      # 마지막 인덱스를 생략하면 디폴트 값은 -1임
array([94, 89, 78, 99])
>>> scores[4:-1]    # 마지막 인덱스로 -1을 사용할 경우 -1의 앞에 있는 78까지 슬라이싱함
array([89, 78])
```

논리적인 인덱싱

- 논리적인 인덱싱 `logical indexing`이란 어떤 조건을 주어서 배열에서 원하는 값을 추려내는 것이다. 예를 들어서 사람들의 나이가 저장된 넘파이 배열 `ages`가 있다고 하자.

```
>>> ages = np.array([18, 19, 25, 30, 28])
```

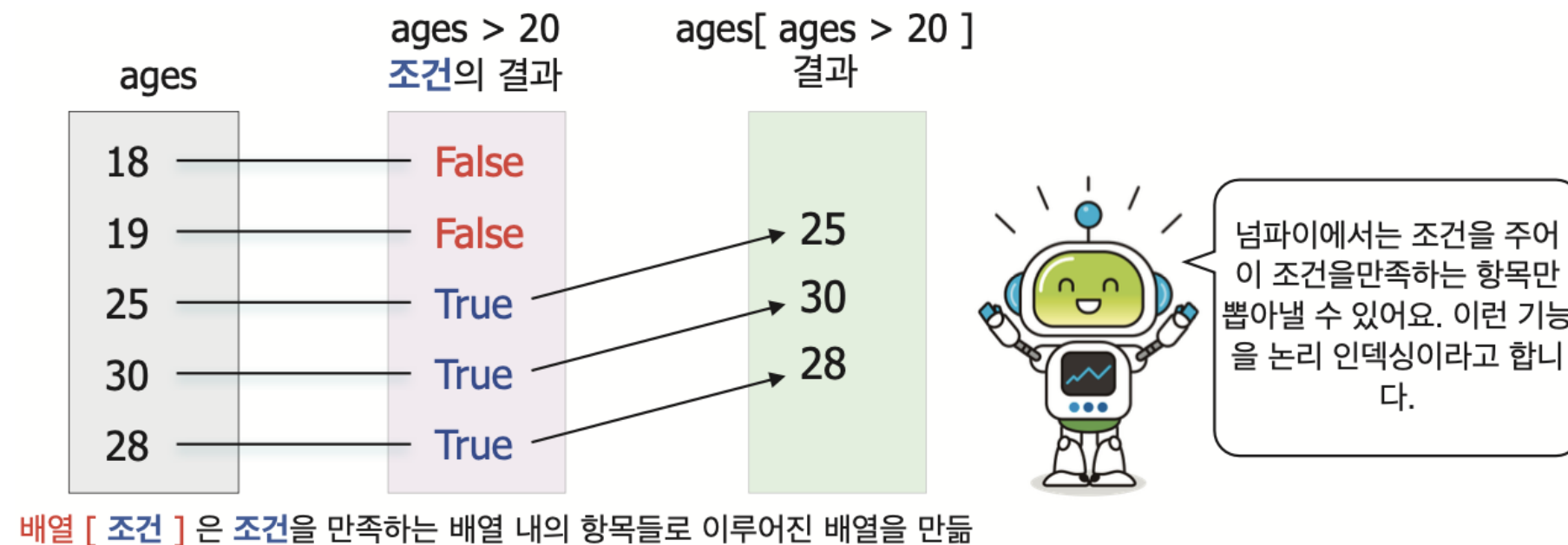
- `ages`에서 20살 이상인 사람만 고르려고 하면 다음과 같은 조건식을 써준다.

```
>>> y = ages > 20
>>> y
array([False, False, True, True, True])
```


논리적인 인덱싱

- 결과는 부울형의 넘파이 배열이 된다.
- ages 배열의 첫 번째와 두 번째 요소는 20보다 크지 않으므로 False가 되고 나머지 요소들은 모두 20보다 크므로 True가 되었다.
- 그런데 실제로는 배열 중에서 20살 이상인 사람들을 뽑아내는 연산이 많이 사용된다.
- 이때는 위의 부울형 배열을 인덱스로 하여 배열 ages에 보내면 된다.

```
>>> ages[ ages > 20 ]  
array([25, 30, 28])
```



2차원 배열 인덱싱

- 넘파이를 사용하면 2차원 배열도 쉽게 만들 수 있다.
- 파이썬의 2차원 리스트는 "리스트의 리스트"라고 할 수 있다.
- 수학에서의 **행렬** `matrix`과는 비슷하지만 리스트는 행렬 연산을 지원하지 않는다.

```
>>> import numpy as np
>>> y = [[1,2,3], [4,5,6], [7,8,9]]      # 2차원 배열(리스트 자료형)
>>> y
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

- 넘파이 2차원 배열은 다음과 같이 `np.array()`를 호출하여 생성할 수 있다.
넘파이의 2차원 배열은 수학에서의 행렬과 같이 다룰 수 있다.

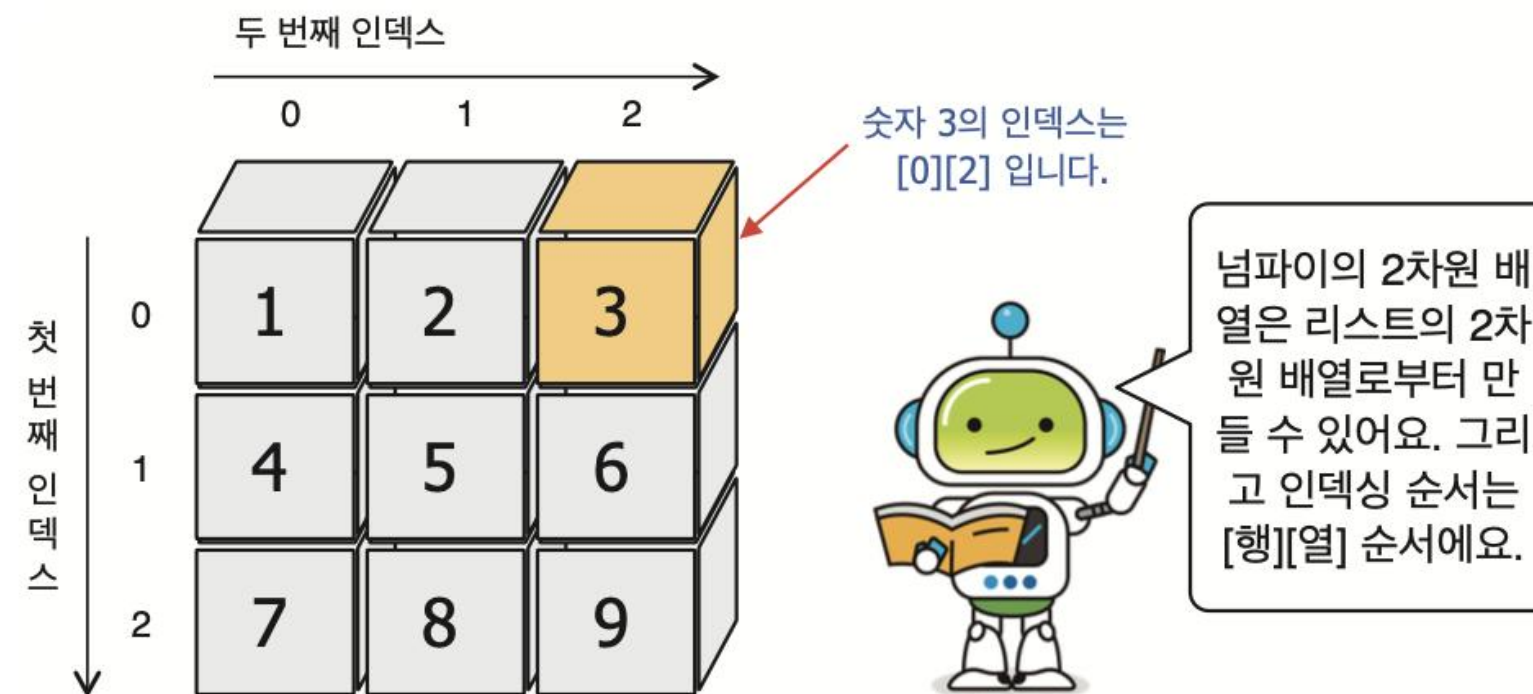
2차원 배열 인덱싱

- 따라서 역행렬이나 행렬식을 구하는 등의 행렬 연산들이 넘파이 배열에 쉽게 적용될 수 있도록 구현되어 있다.

```
>>> np_array = np.array(y)      # 2차원 배열(넘파이 다차원 배열)
>>> np_array
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

2차원 배열 인덱싱

- 2차원 배열에서 특정한 위치에 있는 요소는 어떻게 꺼낼까?
- 2차원 배열도 인덱스를 사용한다. 다만 2차원이기 때문에 인덱스가 2개 필요하다. 첫 번째 인덱스는 행의 번호이다.
- 두 번째 인덱스는 열의 번호이다. 예를 들어서 `np_array[0][2]`는 3이 된다.



```
>>> np_array[0][2]  
3
```

2차원 배열 인덱싱

- 넘파이의 2차원 배열에서 `np_array[0][2]`와 같은 형태로도 특정한 요소를 꺼낼 수 있다.
- 하지만 넘파이에서는 많이 사용되는 표기법이 있다. 0번째 행과 2번째 열에 있는 요소에 접근할 때는 콤마를 사용하여 `np_array[0, 1]`로 써주어도 된다.
- 콤마 앞에 값은 행을 나타내며, 콤마 뒤에 값은 열을 나타낸다.

```
>>> np_array = np.array([[1,2,3], [4,5,6], [7,8,9]])  
>>> np_array[0, 2]    # np_array[0][2]과 동일한 결과  
3
```


2차원 배열 인덱싱

- 2차원 넘파이 배열은 행렬이라는 점을 명심하자. 행렬에서는 행의 번호와 열의 번호만 있으면 특정한 요소를 꺼낼 수 있다.
- 따라서 넘파이 스타일로 [row, col] 인덱스를 사용하면 row 행을 가져온 뒤에 거기서 col 번째 항목을 찾는 것이 아니라 바로 특정 항목에 접근하게 된다.

```
>>> np_array[0, 0]  
1  
>>> np_array[2, -1]  
9
```

2차원 배열 인덱싱

- 그리고 리스트와 유사하게 인덱스 표기법을 사용하여 배열의 요소를 변경할 수 도 있다.

```
>>> np_array[0, 0] = 12    # ndarray의 첫 요소를 12로 변경함
>>> np_array
array([[12, 2, 3],
       [ 4, 5, 6],
       [ 7, 8, 9]])
```

- 파이썬 리스트와 달리, 넘파이 배열은 모든 항목이 동일한 자료형을 가진다는 것을 명심하여야 한다. 예를 들어 정수 배열에 부동 소수점 값을 삽입하려고 하면 소수점 이하값은 자동으로 사라진다.

```
>>> np_array[2, 2] = 1.234  # 마지막 요소의 값을 실수로 변경하려고 하면 실패
>>> np_array
array([[12, 2, 3],
       [ 4, 5, 6],
       [ 7, 8, 1]])
```

2차원 배열 인덱싱

- 넘파이에서 슬라이싱은 큰 행렬에서 작은 행렬을 끄집어내는 것으로 이해하면 된다. 다음은 2차원 행렬의 일부를 추출하는 코드이다.

```
>>> np_array = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]])
>>> np_array[0:2, 2:4]
array([[3, 4],
       [7, 8]])
```

- 위의 표기법은 0에서 1까지의 행에서 2에서 3까지의 열로 이루어진 행렬을 지정한 것이다. 따라서 위와 같은 행렬이 출력된다.
- 넘파이의 2차원 행렬에서 하나의 행을 지정하는 방식은 다음과 같이 할 수 있다.

```
>>> np_array[0]
array([1, 2, 3, 4])
```

- 또한 다음과 같은 넘파이 스타일의 표기법도 가능하다.

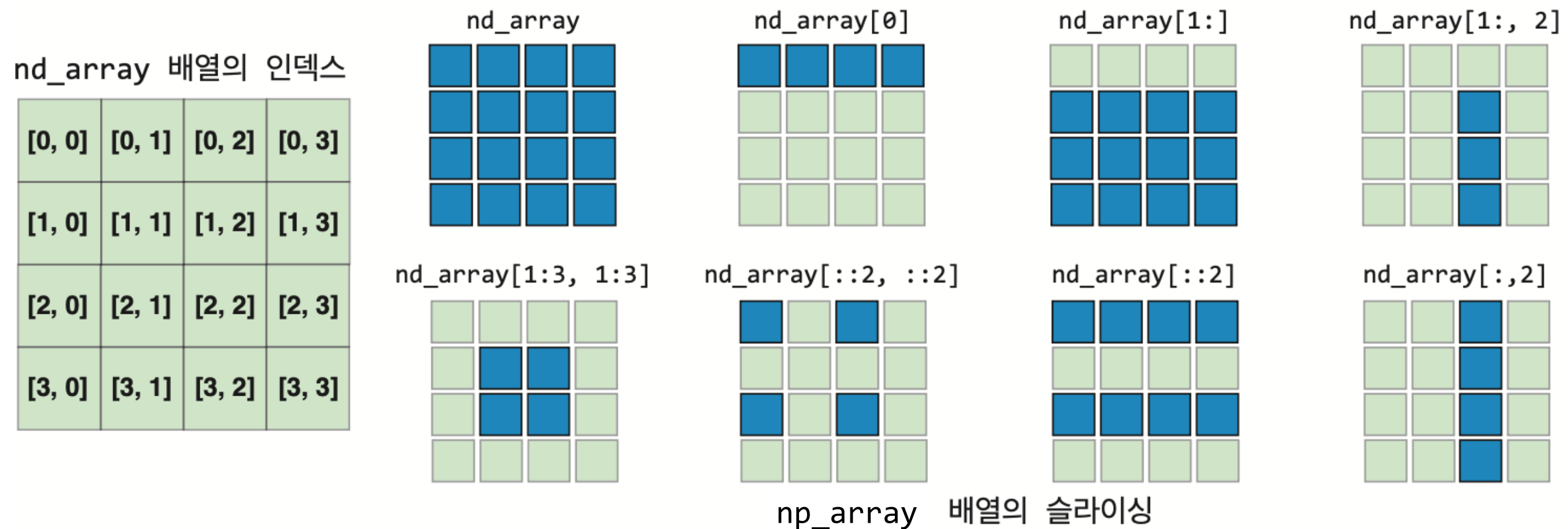
```
>>> np_array[1, 1:3]
array([6, 7])
```


2차원 배열 인덱싱

- 또한 다음과 같은 넘파이 스타일의 표기법도 가능하다.

```
>>> np_array[1, 1:3]  
array([6, 7])
```

- 아래의 그림은 4x4 크기의 np_array라는 ndarray와 이 ndarray의 인덱싱 및 슬라이싱 결과를 보여준다.



2차원 배열 인덱싱



잠깐 - 파이썬 리스트 슬라이싱과 넘파이 스타일 슬라이싱의 차이

다음과 같이 파이썬 리스트 슬라이싱과 넘파이 스타일의 슬라이싱을 적용했을 때, 슬라이싱에 사용된 범위와 간격은 동일하지만 전혀 다른 결과가 나온다. 이 이유를 잘 이해하는 것이 중요하다.

```
np_array = np.array([[ 1,  2,  3,  4],
                     [ 5,  6,  7,  8],
                     [ 9, 10, 11, 12],
                     [13, 14, 15, 16]])
print(np_array[::2][::2]) # 첫 슬라이싱: 0행, 2행 선택, 두 번째 슬라이싱: 그 중 0행 선택
print(np_array[::2,::2]) # 행 슬라이싱: 0행, 2행 선택, 열 슬라이싱: 0열 2열 선택
```

```
[[1 2 3 4]]
[[ 1  3]
 [ 9 11]]
```

2차원 배열에서 논리적 인덱싱

- 2차원 배열에서도 어떤 조건을 주어서 조건에 맞는 값들만 추려낼 수 있다.
- 이 코드는 np_array에 1에서 9까지의 값이 들어있는 2차원 배열에 대해서 `np_array > 5` 계산식의 결과를 보여준다.
- 이 계산의 결과 1, 2, 3, 4, 5까지의 값은 $1 > 5$, $2 > 5$, ..., $5 > 5$ 의 연산의 결과인 False 값이 나타남을 알 수 있다. 나머지 값인 6, 7, 8, 9는 모두 5보다 크기 때문에 True 값이 나타남을 알 수 있다.

```
>>> np_array = np.array([[1,2,3], [4,5,6], [7,8,9]])
>>> np_array > 5
      array([[False, False, False],
            [False, False, True],
```

2차원 배열에서 논리적 인덱싱

- 2차원 배열에 대해서 비교연산자를 사용하면 위와 같이 True, False로 이루어진 배열이 반환되는데, 이것을 이용하여 특정한 값들을 뽑아낼 수도 있다.
- 다음과 같이 코드를 살펴보자. 이 코드를 살펴보면 np_array의 큰 괄호 내부에서 np_array > 5 연산을 적용하였다.
- 이렇게 할 경우 np_array 배열 내의 모든 원소들 중에서 5보다 큰 값만이 추출되어 [6, 7, 8, 9]와 같은 1차원 행렬이 출력되는 것을 볼 수 있다.

```
>>> np_array[ np_array > 5 ]  
array([6, 7, 8, 9])
```

2차원 배열에서 논리적 인덱싱

- 좀 더 나아가 아래와 같이 세번째 열을 추려내는 연산을 살펴보면 세번째 열의 모든 원소 [3, 6, 9]가 나타남을 볼 수 있다.

```
>>> np_array[:, 2]  
array([3, 6, 9])
```

- 이제 이 슬라이싱의 결과 값 [3, 6, 9]중에서 5를 넘는 값이 있는지를 부울형으로 반환한다. 이 경우 간단하게 다음과 같은 연산을 통해서 False, True, True를 얻을 수 있다.

```
>>> np_array[:, 2] > 5  
array([False, True, True])
```


2차원 배열에서 논리적 인덱싱

- 이제 크기 비교 연산자를 약간 수정하여 다음과 같은 짝수 구하기 연산자를 적용해 보자.
- 위에서 살펴본 바와 같이 %2의 결과가 0인 경우가 짝수인 경우에 해당하므로 결과는 짝수 값이 있는 곳만 True가 나타나고 나머지는 모두 False가 나타남을 볼 수 있다.

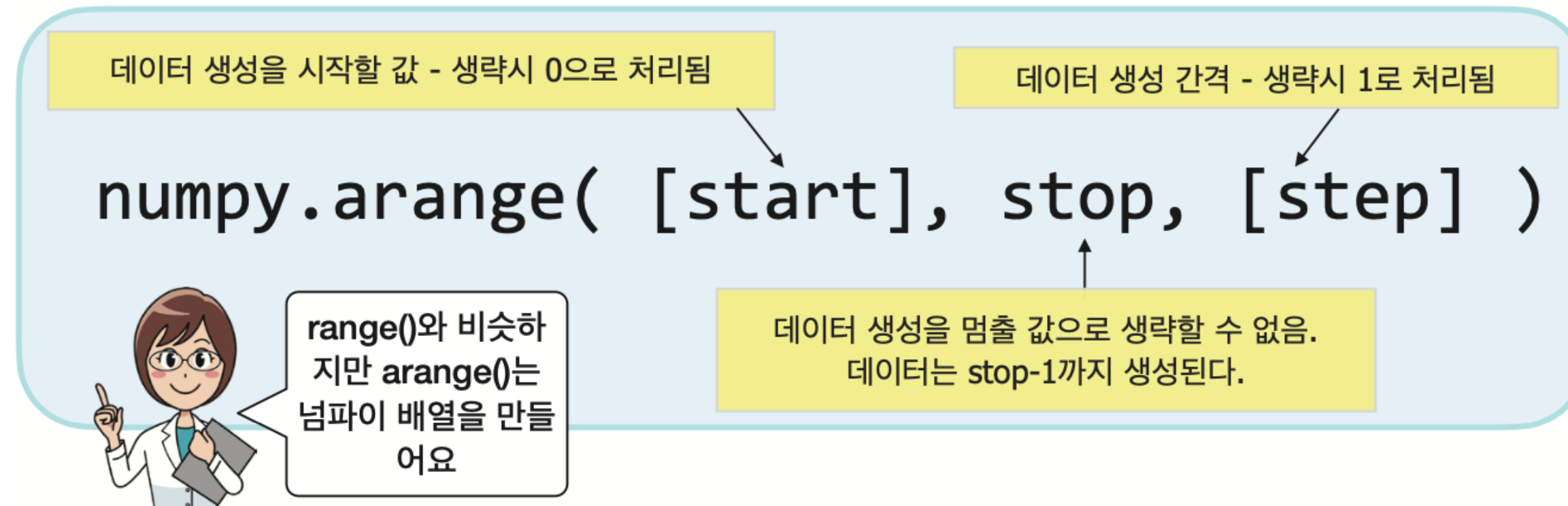
```
>>> np_array % 2 == 0  
array([[False,  True, False],  
       [ True, False,  True],  
       [False,  True, False]])
```

- 이제 이 값이 True인 원소만을 추출하면 손쉽게 다음과 같은 1차원 배열을 얻을 수 있다. 이를 응용하면 특정수의 배수를 추출하는 등의 필터링 작업을 손쉽게 할 수 있다

```
>>> np_array[ np_array % 2 == 0 ]  
array([2, 4, 6, 8])
```

arange() 함수와 range() 함수

- 넘파이도 많은 함수들을 가지고 있다.
- 제일 먼저 살펴볼 함수는 arange() 함수이다.



```
>>> import numpy as np
>>> np.arange(5)           # 넘파이의 arange로 연속적인 정수를 생성하자
array([0, 1, 2, 3, 4])
```

arange() 함수와 range() 함수

- 시작값을 지정하려면 다음과 같이 한다.

```
>>> np.arange(1, 6)      # 1부터 시작되는 연속적인 정수를 생성하자
array([1, 2, 3, 4, 5])
```

- 증가되는 값을 지정하려면 다음과 같이 한다.

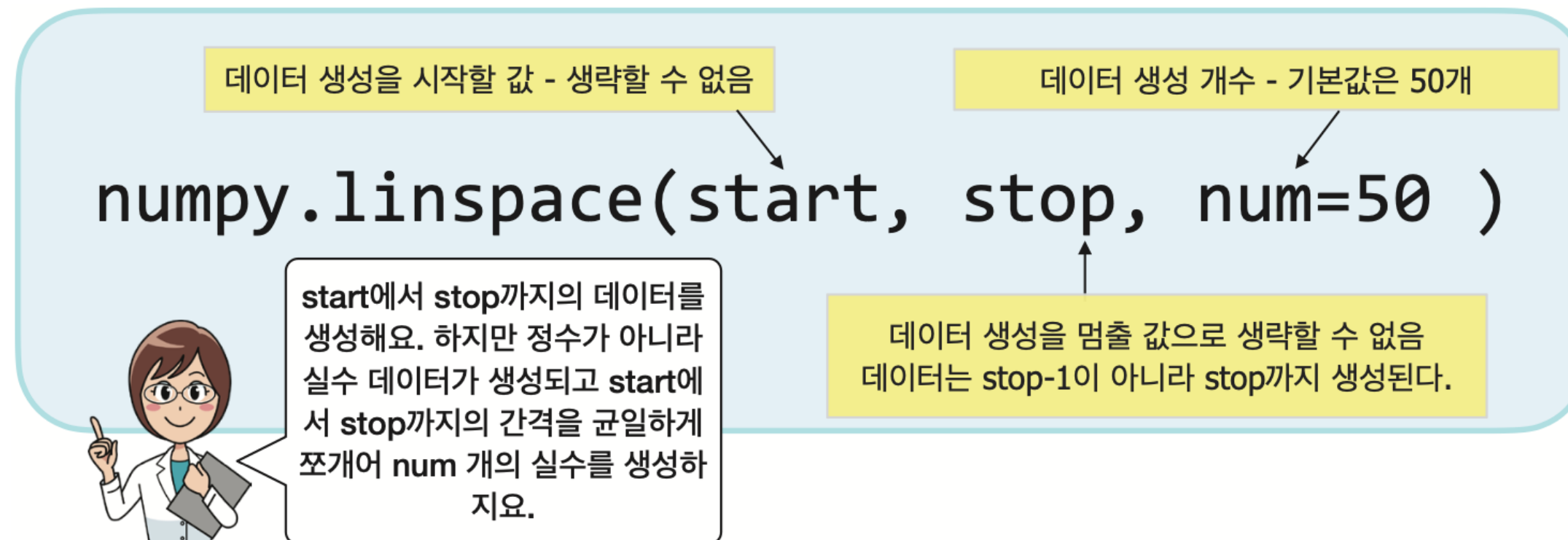
```
>>> np.arange(1, 10, 2)   # arange로 2씩 증가하는 숫자열을 생성하자
array([1, 3, 5, 7, 9])
>>> np.arange(1, 10, 2.5) # 넘파이의 arange는 실수값 step이 가능하다
array([1., 3.5, 6., 8.5])
```

- 이것은 우리가 for 반복문을 위해 많이 사용했던 range() 함수와 비슷하다. 그런데 range()를 통해 생성된 것은 **반복 가능 객체** `iterable` 이고 이것으로 리스트를 만들 수 있다.

```
>>> range(5)
range(0, 5)
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> np.array(range(5))    # range() 함수로 넘파이 다차원 배열을 생성
array([0, 1, 2, 3, 4])
```

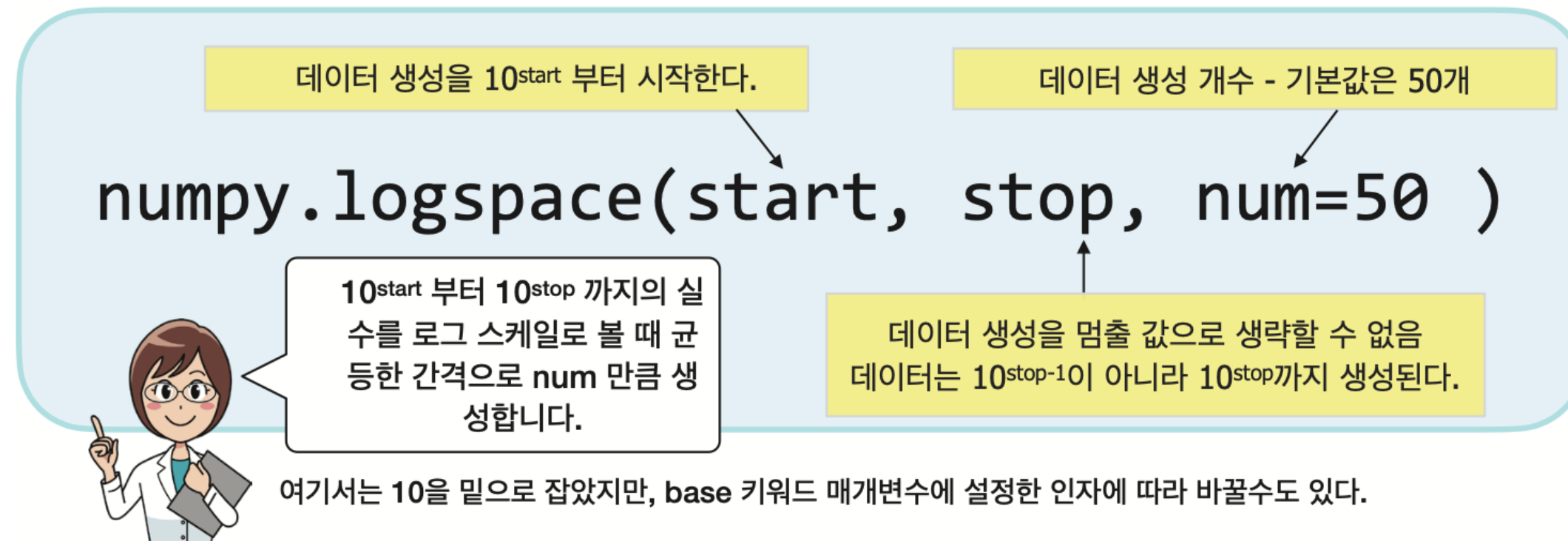

linspace() 함수와 logspace() 함수

- linspace()는 상당히 많이 사용되는 함수이다. linspace()는 시작값부터 끝값까지 균일한 간격으로 지정된 개수만큼의 배열을 생성한다.



linspace() 함수와 logspace() 함수

- 비슷한 함수로 logspace() 함수가 있다. 이것은 로그 스케일로 수들을 생성한다.



linspace() 함수와 logspace() 함수

```
>>> np.linspace(0, 10, 100)
array([ 0.          ,  0.1010101 ,  0.2020202 ,  0.3030303 ,  0.4040404 ,
        ...
        8.08080808,  8.18181818,  8.28282828,  8.38383838,  8.48484848,
        8.58585859,  8.68686869,  8.78787879,  8.88888889,  8.98989899,
        9.09090909,  9.19191919,  9.29292929,  9.39393939,  9.49494949,
        9.5959596 ,  9.6969697 ,  9.7979798 ,  9.8989899 , 10.          ])
```

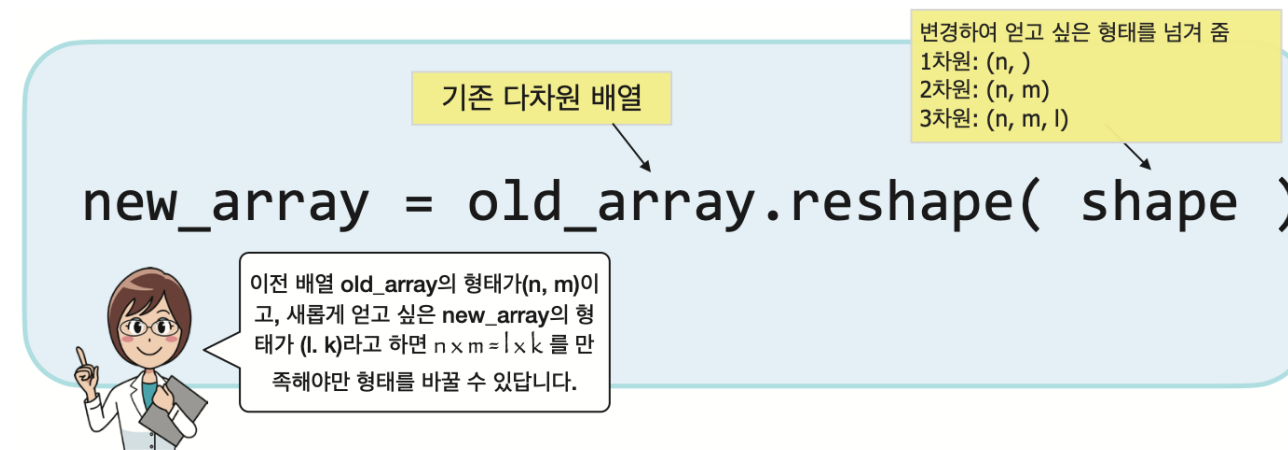
linspace(0, 10, 100)이라고
호출하면 0에서 10까지 총
100개의 수들이 생성

```
>>> np.logspace(0, 5, 10)
array([1.00000000e+00,  3.59381366e+00,  1.29154967e+01,  4.64158883e+01,
        1.66810054e+02,  5.99484250e+02,  2.15443469e+03,  7.74263683e+03,
        2.78255940e+04,  1.00000000e+05])
```

logspace(x, y, n) : 생성되는 수의
시작은 10^x 부터 10^y 까지가 되며,
n개의 수가 생성

reshape() 메소드와 flatten() 메소드

- reshape() 함수는 상당히 많이 사용되는 함수이다. 데이터의 개수는 유지한 채로 배열의 차원과 형태를 변경한다. 이 함수의 인자인 shape을 튜플의 형태로 넘겨주는 것이 원칙이지만 reshape(x, y)라고 하면 reshape((x, y))와 동일하게 처리된다.



```
>>> y = np.arange(12)
>>> y
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
>>> y.reshape(3, 4)
array([[ 0, 1, 2, 3],
       [ 4, 5, 6, 7],
       [ 8, 9, 10, 11]])
```

reshape() 메소드와 flatten() 메소드

```
>>> y.reshape(6, -1)
array([[ 0,  1],
       [ 2,  3],
       [ 4,  5],
       [ 6,  7],
       [ 8,  9],
       [10, 11]])
```

인수로 -1을 전달하면 데이터의 개수에 맞춰서 자동으로 배열의 형태가 결정

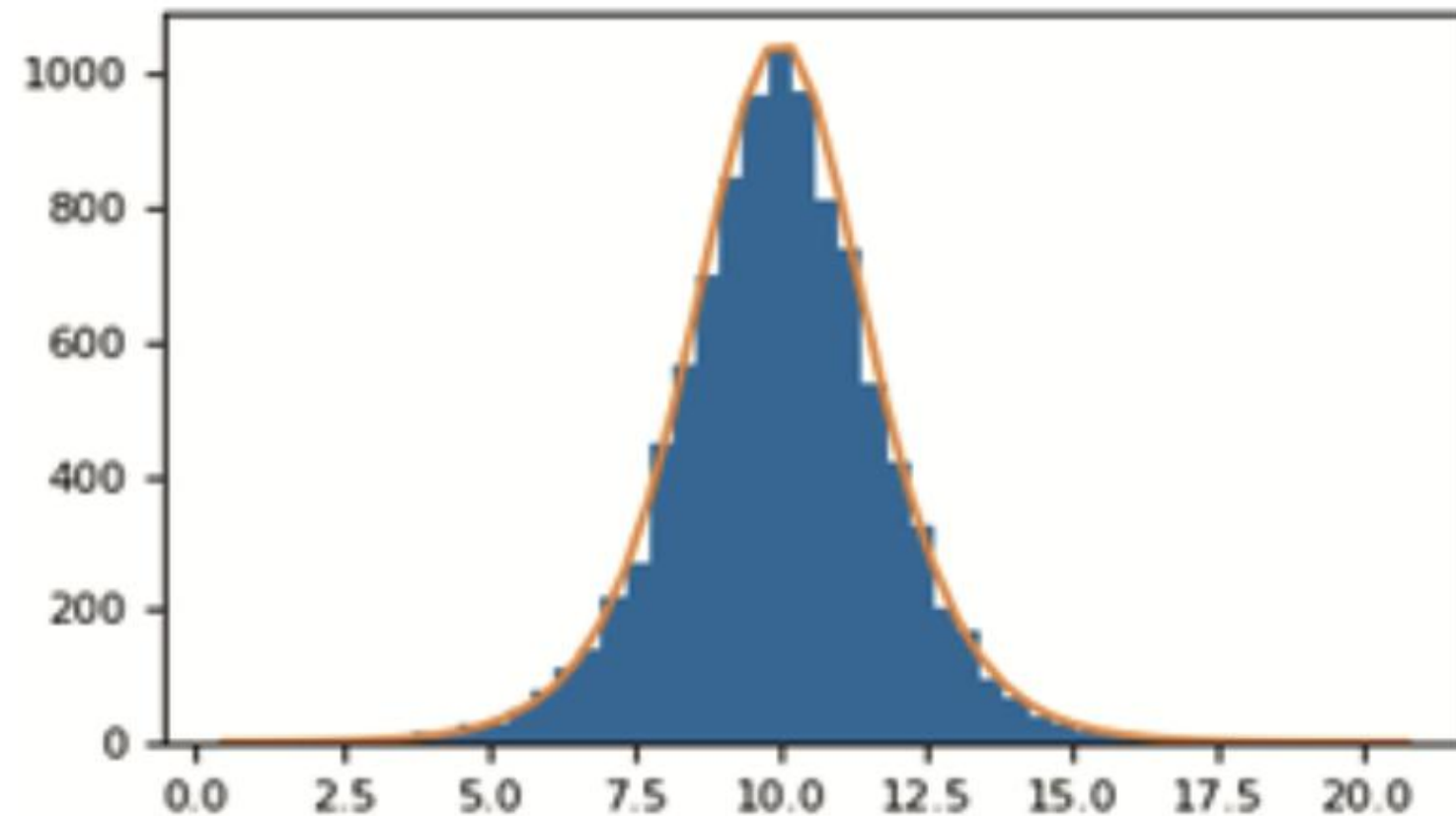
```
>>> y.reshape(7, 2)
...
y.reshape(7, 2)
ValueError: cannot reshape array of size 12 into shape (7,2)
```

```
>>> y.flatten()    # 2차원 배열을 1차원 배열로 만들어 준다
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

flatten()은 평탄화 함수로 2차원 이상의 고차원 배열을 1차원 배열로 만들어 준다.

난수의 생성

- 데이터 과학자가 다룰 데이터는 상당히 크며, 몇 십만 개 이상의 데이터를 다루어야 하는 경우도 많다.
- 10,000개 이상의 임의의 데이터를 어떻게 생성할 것인가?
- 직접 10,000개 이상의 데이터를 입력할 수도 있지만 어떤 확률 분포에서 난수를 생성하여서 실험 데이터로 사용할 수도 있을 것이다.



난수의 생성



잠깐 - 난수란?

난수 random number는 **무작위성** randomness의 특징을 가지고 출현하는 수를 의미한다. 무작위성이라는 것은 특정한 패턴이 존재하지 않아 다음에 어떤 수가 나타날지 예측할 수 없다는 것이다.

자연에서 진짜 난수를 얻어내는 방법은 쉽지 않다. 대기나 열에서 발생하는 잡음, 양자 현상과 같은 무작위성을 띤 현상을 측정하여 편향을 보정하는 방법을 사용하기도 한다. 우주 배경 복사도 진짜 난수를 발생시키는 원천 데이터로 사용할 수 있다.

난수가 필요할 때마다 이런 복잡한 일을 수행하기는 쉽지 않다. 그래서 일반적으로 컴퓨터 프로그래밍에서는 **의사 난수** pseudo random number를 사용한다. 이것은 **시드** seed라고 하는 난수 발생의 씨앗이 될 수를 주면, 이 값을 가지고 예측하기 어려운 수를 생성해 내는 것이다. 따라서 이 난수는 동일한 시드가 주어진다면 동일한 난수를 생성하게 된다.

난수의 생성

- 넘파이에서 난수의 **시드**seed를 설정하는 문장은 다음과 같다.

```
>>> np.random.seed(100) # 난수를 만들기 위한 초기값
```

- 시드가 설정되면 다음과 같은 문장을 수행하여 5개의 난수를 얻을 수 있다.
- 난수는 컴퓨터가 규칙을 가지고 생성한 수로 정확한 의미의 난수는 아니지만 난수에 가까운 수로 **의사난수**pseudo random number라고 부른다.

난수의 생성

```
>>> np.random.rand(5)
array([0.54340494, 0.27836939, 0.42451759, 0.84477613, 0.00471886])
```

```
>>> np.random.rand(5, 3)
array([[0.12156912, 0.67074908, 0.82585276],
       [0.13670659, 0.57509333, 0.89132195],
       [0.20920212, 0.18532822, 0.10837689],
       [0.21969749, 0.97862378, 0.81168315],
       [0.17194101, 0.81622475, 0.27407375]])
```

난수로 이루어진 2차원
배열(5x3)

```
>>> a = 10
>>> b = 20
>>> (b - a) * np.random.rand(5) + a
array([14.31704184, 19.4002982 , 18.17649379, 13.3611195 , 11.75410454])
```

10에서 20 사이에 있는 난수
5개 생성

```
>>> np.random.randint(1, 7, size=10)
array([4, 3, 4, 1, 1, 2, 6, 6, 2, 6])
```

```
>>> np.random.randint(1, 11, size=(4, 7))
array([[10, 2, 6, 9, 8, 5, 3],
       [ 7, 3, 2, 9, 5, 3, 2],
       [ 3, 1, 6, 2, 9, 8, 2],
       [ 7, 5, 2, 8, 3, 3, 6]])
```

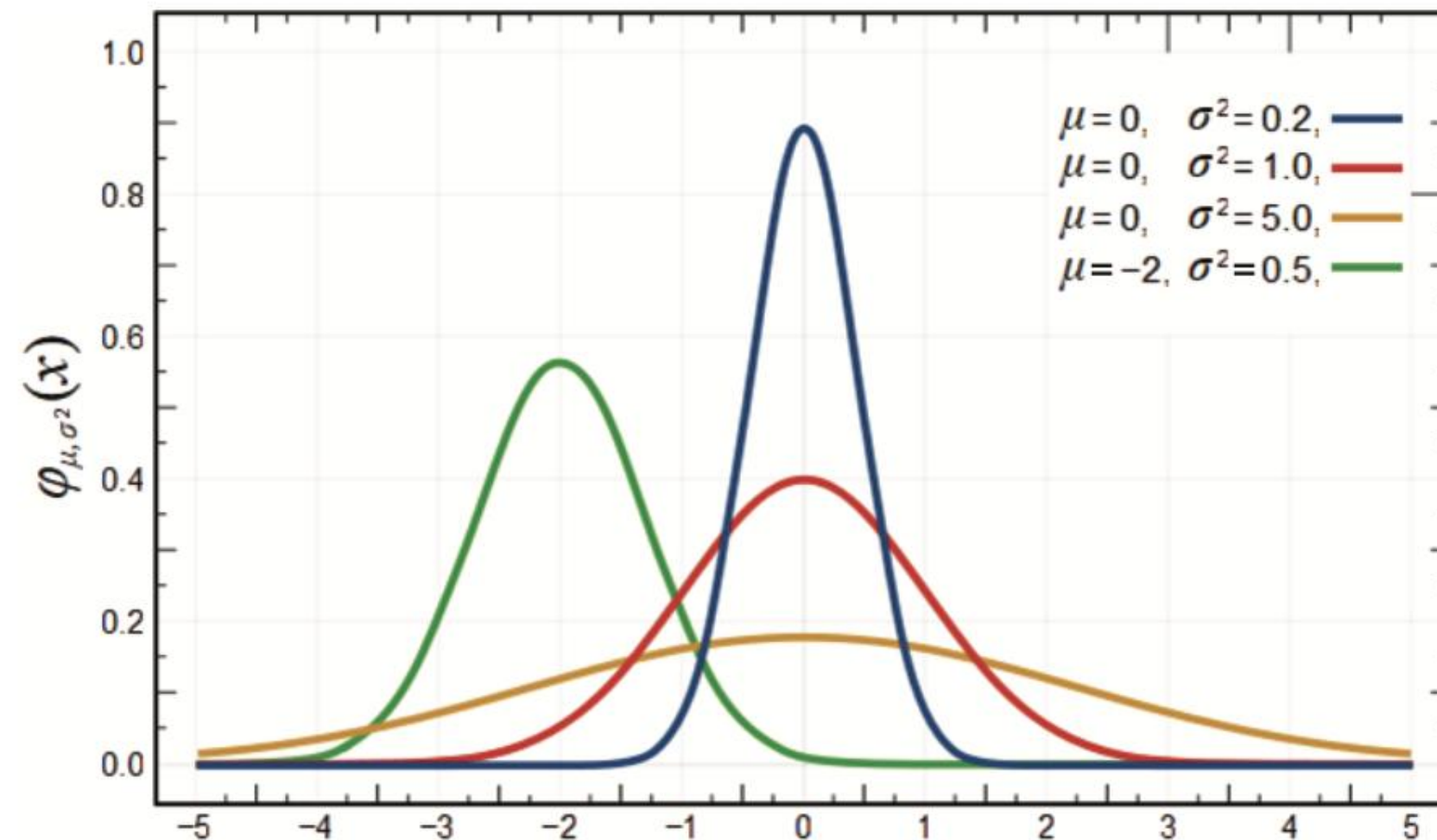
1부터 (11-1)=10
사이의 4행 7열
난수 생성



컴퓨터가 최대한 난
수에 가깝게 만든 난
수를 의사난수라고
한답니다.

난수의 생성

- 앞에서 생성한 난수는 균일한 확률 분포로 생성된다.
- 확률분포는 평균값에서 가장 높고 평균값에서 멀수록 발생확률이 낮아진다.
- 표준편차(σ)가 크면 클수록 데이터의 흩어짐이 크기 때문에 발생 확률이 평평하게 퍼진 상태에 접근하게 된다.



난수의 생성

```
>>> np.random.randn(5)
array([ 0.78148842, -0.65438103, 0.04117247, -0.20191691, -0.87081315])
```

```
>>> np.random.randn(5, 4)
array([[ 0.22893207, -0.40803994, -0.10392514, 1.56717879],
       [ 0.49702472, 1.15587233, 1.83861168, 1.53572662],
       [ 0.25499773, -0.84415725, -0.98294346, -0.30609783],
       [ 0.83850061, -1.69084816, 1.15117366, -1.02933685],
       [-0.51099219, -2.36027053, 0.10359513, 1.73881773]])
```

난수로 이루어진 2차원 배열
(5행 4열의 난수)

```
>>> mu = 10
>>> sigma = 2
>>> randoms = mu + sigma * np.random.randn( 5, 4 )
>>> randoms
array([[ 9.82507212,  7.12282389,  5.88878504,  7.5865665 ],
       [ 6.05953536, 11.73521791, 10.90362868, 12.33878255],
       [10.2980491 ,  8.64563344,  9.09398278, 11.20863908],
       [ 9.30825873,  9.81230228,  9.71131179, 12.47776473],
       [10.00162592,  9.86745157,  8.51138086,  9.82922367]])
```

평균이 10이고 표준편차 값이
2인 정규분포를 가지는 난수

정규 분포 난수 생성

- 10,000명의 키를 난수로 생성해보자. 평균은 175cm이고 표준편차는 10인 정규분포를 따르게 하겠다.

```
>>> m = 175
>>> sigma = 10
>>> heights = m + sigma * np.random.randn(10000)
>>> heights
array([165.64438142, 172.90778856, 207.82109029, ..., 163.88596647,
       177.29401299, 180.7002071 ])
```

정규 분포 난수 생성

- 위의 데이터를 받아서 평균과 중앙값을 계산해 보자.

```
>>> np.mean(heights)
175.14185004766918
```

```
>>> np.median(heights)
175.0251183448534
```

중앙값 계산: 리스트의 중앙에 있는 항목

```
>>> a = np.array([ 3, 7, 1, 2, 21]) # 21은 전체 데이터 중에서 비정상적으로 큰 값이다
>>> np.mean(a)
6.8
>>> np.median(a) # [3, 7, 1, 2, 21]들 중 가운데 항목을 구한다
3.0
```


상관관계 계산하기

- 주변에는 일반적으로 키가 큰 사람의 몸무게가 키가 작은 사람에 비해서 많이 나가는 것을 볼 수 있다.
- `corrcoef(x, y)` 함수는 요소들의 상관관계를 계산한다.
- 이때 `x`와 `y`는 데이터를 담고 있는 리스트나 배열이 될 수 있다.
- 결과는 `x`와 `x`의 상관관계, `x`와 `y`의 상관관계, `y`와 `x`의 상관관계를, 그리고 `y`와 `y`의 상관관계를 라고 할 때, 다음과 같은 행렬을 반환한다.

$$\begin{bmatrix} C_{xx} & C_{xy} \\ C_{yx} & C_{yy} \end{bmatrix}$$

상관관계 계산하기

```
x = [ 1, 2, 3, 4, ..., 97, 98, 99 ]  
y = [ 1, 4, 9, 16, ..., 9409, 9604, 9801]
```

x값이 증가하면 y값도 증가
둘 사이의 상관관계는 매우 높음

```
import numpy as np
```

```
x = [ i for i in range(100) ]      # 0에서 99까지의 값을 요소로 하는 리스트  
y = [ i ** 2 for i in range(100) ] # 0에서 99까지의 값의 제곱을 요소로 하는 리스트
```

```
result = np.corrcoef(x, y)  
print(result)
```

```
[[1.          0.96764439]  
 [0.96764439 1.          ]]
```


상관관계 계산하기



잠깐 - corrcoef() 함수가 계산하는 상관관계의 수학적 정의

넘파이의 `corrcoef()` 함수는 수학적으로 **피어슨** **Pearson** 상관 계수를 계산하는 함수이다. 피어슨 상관 계수는 통계학에서 사용하는 상관 계수로 두 변량의 공분산을 각각의 표준편차를 서로 곱한 값으로 나눈 것이다. 상세한 것은 통계학 교과서를 참고하도록 하자.

상관관계 계산하기

- 보다 많은 수의 변수를 사용하여 상관관계를 계산할 수도 있다.
- 예를 들어 다음과 같이 세 개의 리스트가 있다고 하자. 각각의 리스트는 100개의 항목을 담고 있다. 이 값들이 서로 어떤 상관관계를 갖는지 알고 싶다.

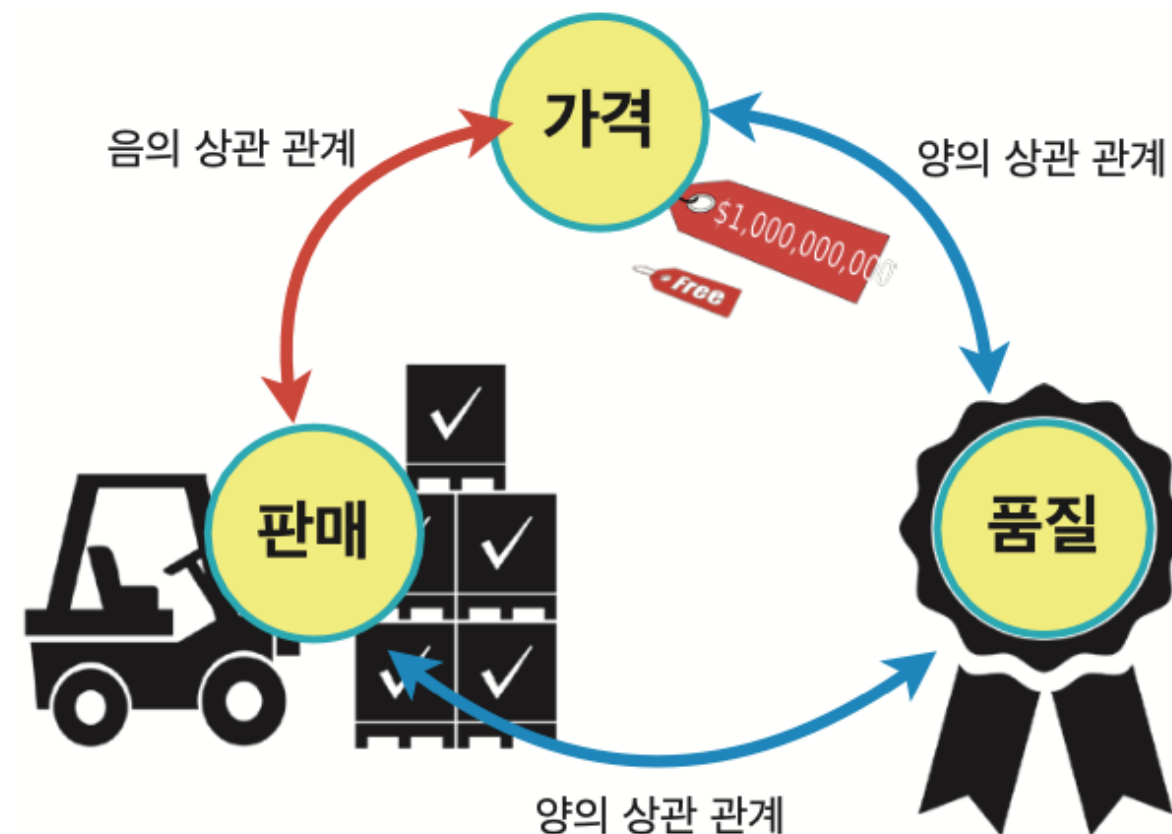
```
x = [ i for i in range(100) ]  
y = [ i ** 2 for i in range(100) ]  
z = [ 100 * np.sin(3.14*i/100) for i in range(100) ]
```

- 새롭게 추가된 리스트 z는 진폭이 100인 **사인**sine 함수로 0도에서 180도까지 그려지게 된다.

상관관계 계산하기

- 변수 a와 b사이의 상관관계를 C_{ab} 라고 하면, 세 개의 변수 x, y, z의 상관관계를 모두 표시하는 행렬은 크기의 다음과 같은 행렬이면 된다. 상관계수는 비선형적인 함수에 대해서는 적용이 어렵다.

$$\begin{bmatrix} C_{xx} & C_{xy} & C_{xz} \\ C_{yx} & C_{yy} & C_{yz} \\ C_{zx} & C_{zy} & C_{zz} \end{bmatrix}$$

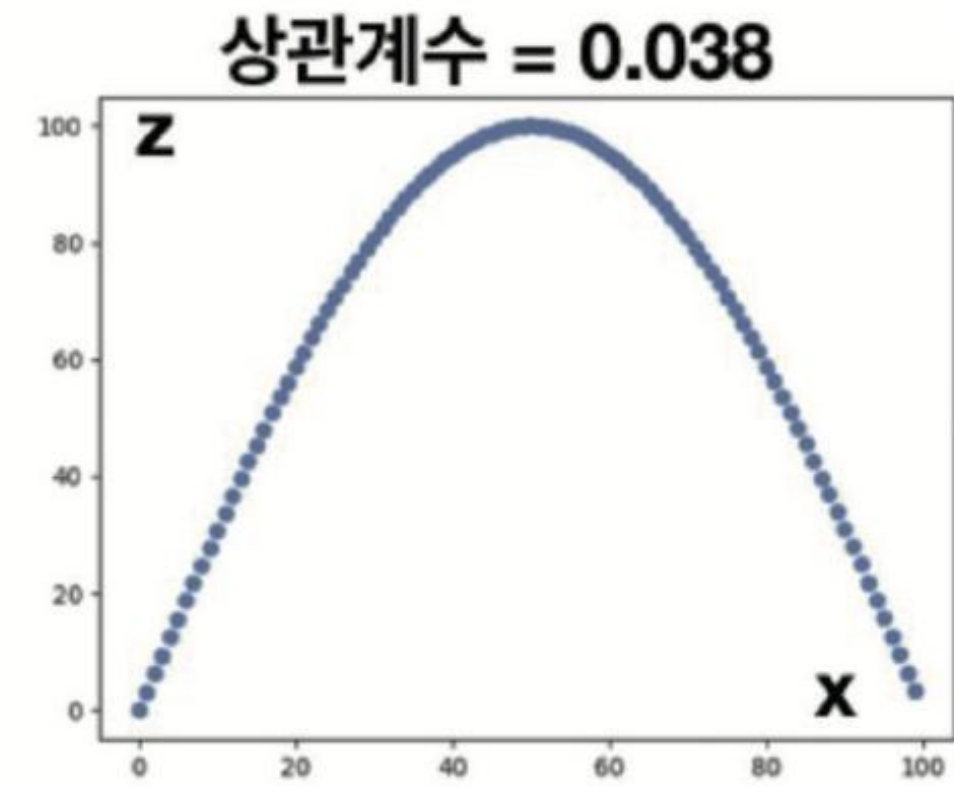
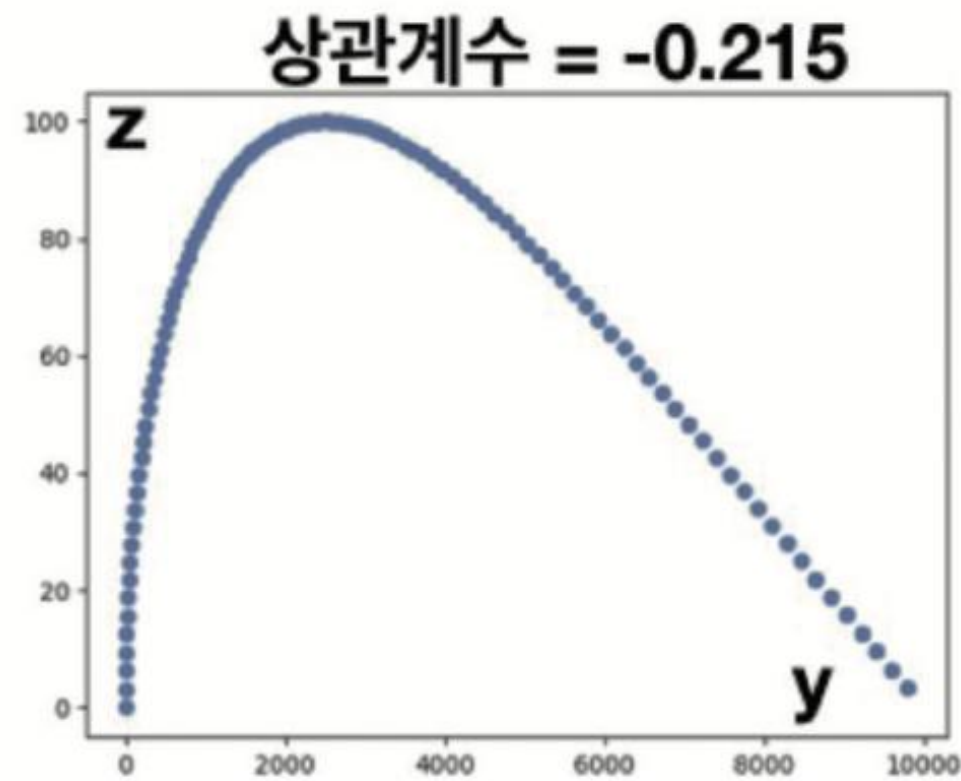
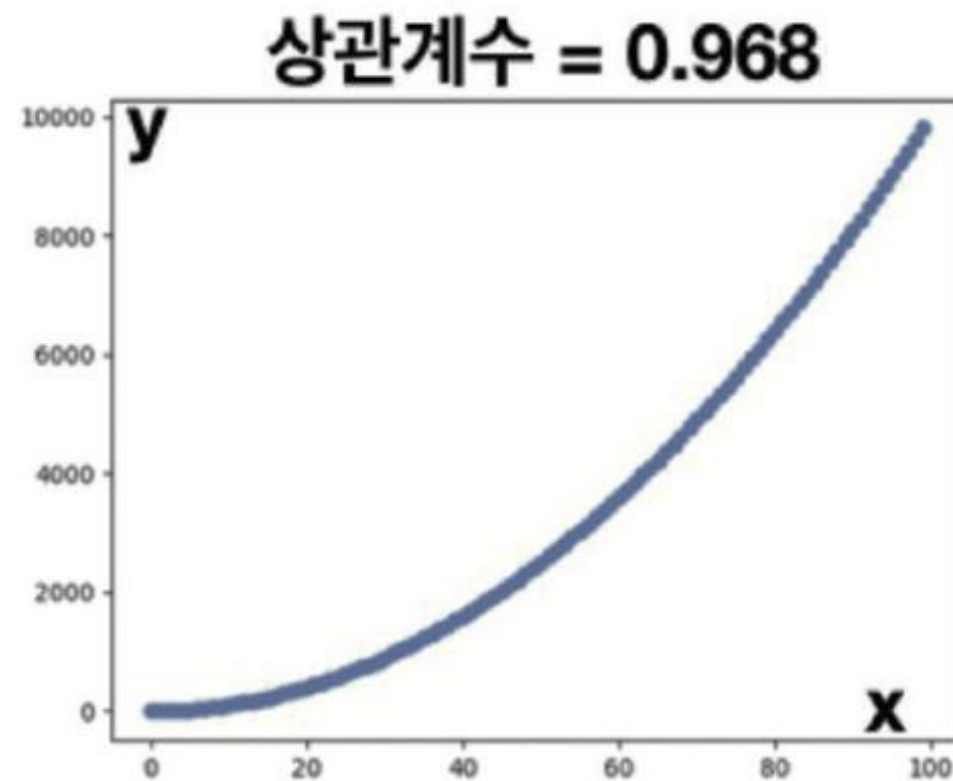


상관관계 계산하기

```
result = np.corrcoef( [x, y, z] )  
print(result)
```

리스트들을 리스트로 묶어서 호출

```
[[ 1.          0.96764439  0.03763255]  
 [ 0.96764439  1.         -0.21532645]  
 [ 0.03763255 -0.21532645  1.          ]]
```



핵심정리

- 넘파이는 리스트가 아니라 배열로 다차원 데이터를 다룬다.
- 배열은 동일한 자료형을 연속해서 가진 데이터 구조로 읽고 쓰는 일을 빠르게 할 수 있다.
- 리스트와 달리 넘파이 배열들을 서로 연산하면 벡터와 행렬처럼 다루어진다.
- 넘파이 배열도 리스트와 다른 반복가능 객체처럼 슬라이싱을 적용할 수 있다.
- 넘파이는 넘파이만의 스타일로 슬라이싱을 할 수 있으며, 리스트 등과 다른 방식의 결과가 나온다.
- 슬라이싱과 별개로 논리적 조건을 부여하여 항목을 가져오는 일도 가능하다.

감사합니다

Jinu Gong
